SYSTEM ANALYSIS DOCUMENTATION (SAD)

UMHackathon 2026

umhackathon@um.edu.my

© UMHackathon 2026

Email: umhackathon@um.edu.my

Table of Content

Contents

1. Introduction: 3
   1.1 Purpose: 3
   1.2 Background: 4
   1.3 Target Stakeholder 5
2. System Architecture & Design 6
   2.1 High Level Architecture 7
   2.2 Technological Stack 14
   2.3 Key Data Flows 19
3. Functional Requirements & Scope 21
   3.1 Minimum Viable Product: 21
   3.2 Non-Functional Requirements (NFRs) 22
   3.3 Out of Scope / Future Enhancements 23
4 Monitor, Evaluation, Assumptions & Dependencies 24
   4.1 Technical Evaluation: 24
   4.2 Monitoring: 29
   4.3 Assumptions: 30
   4.4 External Dependencies: 31
5. Project Management & Team Contributions 32

© UMHackathon 2026

Email: umhackathon@um.edu.my

1. Introduction:

LOOM is an AI-powered autonomous business operating system that transforms business ideas into fully operational businesses in under 10 minutes through conversational setup, intelligent automation, and multi-agent decision-making. The system consolidates business onboarding, customer service, order management, marketing intelligence, and strategic planning into a single unified platform.

This chapter introduces LOOM's purpose, provides business context on the problem it solves, and identifies all key stakeholders and their expectations. It establishes the foundation for understanding the technical architecture and design decisions detailed in subsequent chapters.

1.1 Purpose:

This System Analysis Document (SAD) provides the technical blueprint for LOOM, an AI-powered autonomous business operating system.

It serves as reference documentation for:

- How LOOM transforms business ideas into operational systems
- Technical design decisions and architectural patterns
- Data flows and system interactions
- AI/LLM integration approach
- End-to-end workflows for core features
- Deployment and scalability strategy

Key Audience:

- Backend & Frontend Developers
- AI/ML Engineers
- QA & Testing Teams
- Project Managers
- DevOps & Infrastructure Teams

© UMHackathon 2026

Email: umhackathon@um.edu.my

1.2 Background:

Small business owners and entrepreneurs struggle to run complete businesses due to:

- Manual handling of customer communication, order tracking, and marketing
- Lack of strategic decision-making support
- Multiple disconnected tools required for operations
- High barrier to entry for non-technical founders

LOOM's Solution

LOOM consolidates business setup, AI-driven operations, marketing intelligence, and strategic planning into one unified autonomous system, reducing friction from idea to execution.

Previous State / Current Market

Aspect	Traditional Approach	LOOM Approach
Setup	Manual configuration, days/weeks	Conversational onboarding, 5-10 minutes
Tools Required	5-7 different SaaS platforms	Single integrated platform
Customer Services	Manual responses (limited hours).	AI-powered 24/7 with escalation
Marketing	Founder creates content	AI generates content & insights
Decision Making	Owner intuition only	Data-driven multi-agent board

Major Architectural Components (LOOM)

Conversational Onboarding System - Natural language business setup

AI Customer Interaction Engine - Autonomous chat with escalation detection

Multi-Agent Board - Strategic decision-making (CEO/CFO/CMO)

Marketing Intelligence System - Content and insight generation

Telegram Integration - Multi-channel AI chat

Real-time Dashboard - Business metrics and operations management

Public Storefront - Customer product browsing and ordering

New Capabilities in LOOM v2.0

Multi-agent reasoning for business strategy

Context-aware AI decisions based on real business data

Automated escalation detection for critical customer issues

Streaming responses for real-time agent feedback

Telegram AI chat integration (chat-only, no ordering)

© UMHackathon 2026

Dynamic marketing content generation (posts, insights, campaigns)

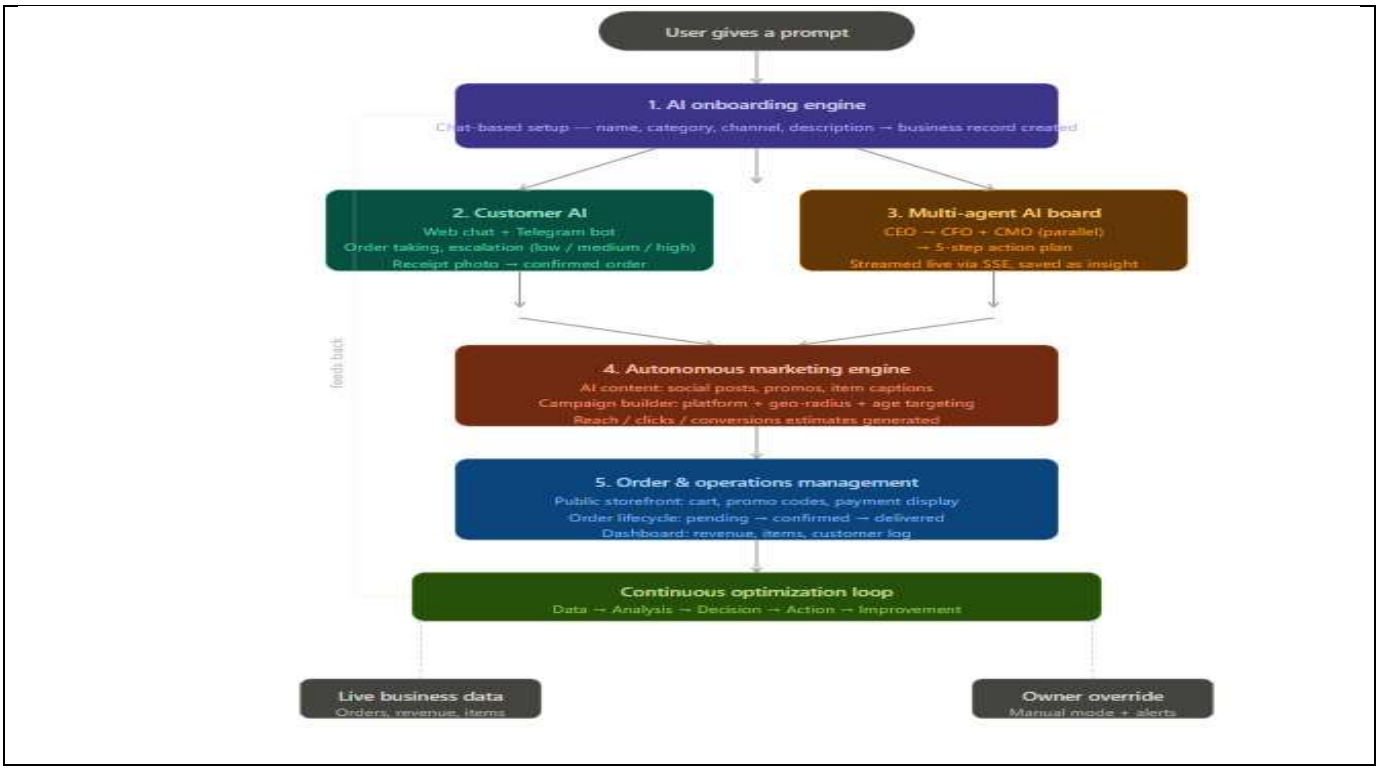
Unified order and customer management dashboard

1.3 Target Stakeholder

Stakeholders	**Roles**	**Expectations**	**Interaction Points**
Business Owner	End user, makes business decisions	Simple setup, hands-off operations, strategic guidance, real-time visibility	Onboarding chat, Dashboard, Multi-agent board, Marketing insights
Customer (End User)	Browses products, places orders, asks questions	Smooth shopping experience, quick responses to inquiries, transparent order status	Public Storefront, Web Chat, Telegram Bot
Admin/Developer	System maintenance and monitoring	Reliable APIs, clear contracts, modular architecture, easy debugging	API routes, Database, Monitoring dashboard
QA/Testing Team	Validates system behavior across features	Defined scope, clear request/response formats, edge case coverage	API contracts, test endpoints, database queries
Marketing Operator	Generates promotional content	Easy-to-use content generation, actionable insights	Marketing Intelligence API, Dashboard insights
System Architect	Ensures scalability and reliability	Cloud-native design, horizontal scaling, failover mechanisms	Infrastructure, Deployment pipeline, Monitoring

© UMHackathon 2026

2. System Architecture & Design



This chapter describes the technical architecture of LOOM, detailing how all system components interact to deliver autonomous business operations. It covers the high-level system design, integration of the GLM language model as a core service layer, dependency relationships between components, and end-to-end user interaction flows through sequence diagrams.

© UMHackathon 2026

Email: umhackathon@um.edu.my

2.1 High Level Architecture

© UMHackathon 2026

Email: umhackathon@um.edu.my

1.1.1. **Overview:**

Type	Details
System	Web-based SaaS Platform + AI Intelligence Layer
Architecture	Cloud-Server with Microservice-like API Routes

System Description:

LOOM is primarily structured as a client-server-based system in which there are 3 types of clients:

1. Business Owner Dashboard (Authenticated Web App) - Manages business operations
2. Public Storefront (Anonymous Web App) - Customer product browsing and ordering
3. Telegram Bot Interface (Messaging) - AI chat for customer inquiries

All these clients communicate via a Backend API layer implemented as Next.js API routes. The backend orchestrates three independent workflows:

1. Onboarding Service - Conversational business setup via GLM
2. Customer Interaction Service - Chat, escalation, and inquiry handling
3. Marketing & Strategy Service - Insights, content generation, multi-agent planning

All services are deployed on Vercel (serverless) and share a PostgreSQL database hosted on Supabase with Row-Level Security for multi-tenant data isolation.

2.1.2 LLM as Service Layer

The ILMU GLM 5.1 language model is integrated as a core service layer (not a generic AI module) that powers multiple autonomous workflows:

GLM Service Integration Points:

1. Onboarding Service - Processes business setup conversation and extracts structured data
2. Customer Chat Service - Generates contextual responses with business awareness and escalation detection

© UMHackathon 2026

3. Marketing Service - Generates insights, social media posts, and promotional content
4. Strategy Service - Executes multi-agent reasoning (CEO/CFO/CMO) for business recommendations

GLM Characteristics:

- API Endpoint: https://api.ilmu.ai/v1/chat/completions
- Authentication: Bearer token (ILMU_API_KEY)
- Model: ILMU GLM 5.1
- Max Tokens: 1024-4096 (per operation)
- Temperature: 0.7-0.8 (balance creativity and consistency)
- Timeout: 90 seconds

Core Functionality:

- Receives structured prompts with business context
- Processes context-aware requests
- Returns text responses or structured JSON
- Supports streaming for real-time feedback
- Integrated error handling with graceful fallbacks

2.1.3 Dependency Diagram

This diagram shows:

a. How prompts are constructed and sent to GLM API

b. What goes into context window

c. How system receives/parses responses

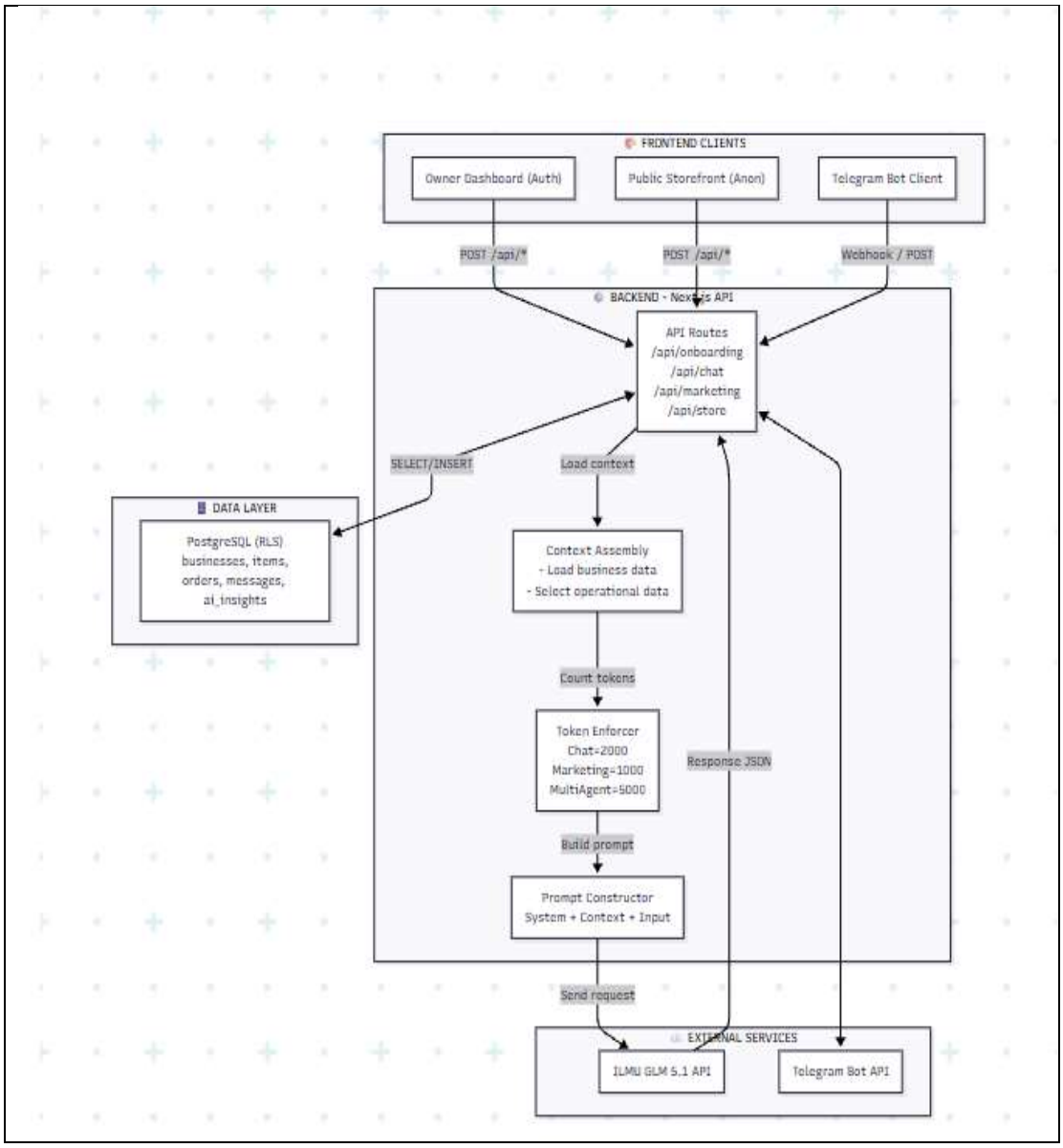
d. Where token limitations are enforced

e. Major API calls between components

f. Frontend, backend, services, database, external APIs interactions

g. Distributed components deployment

© UMHackathon 2026



© UMHackathon 2026

Email: umhackathon@um.edu.my

2.1.4 Sequence Diagram

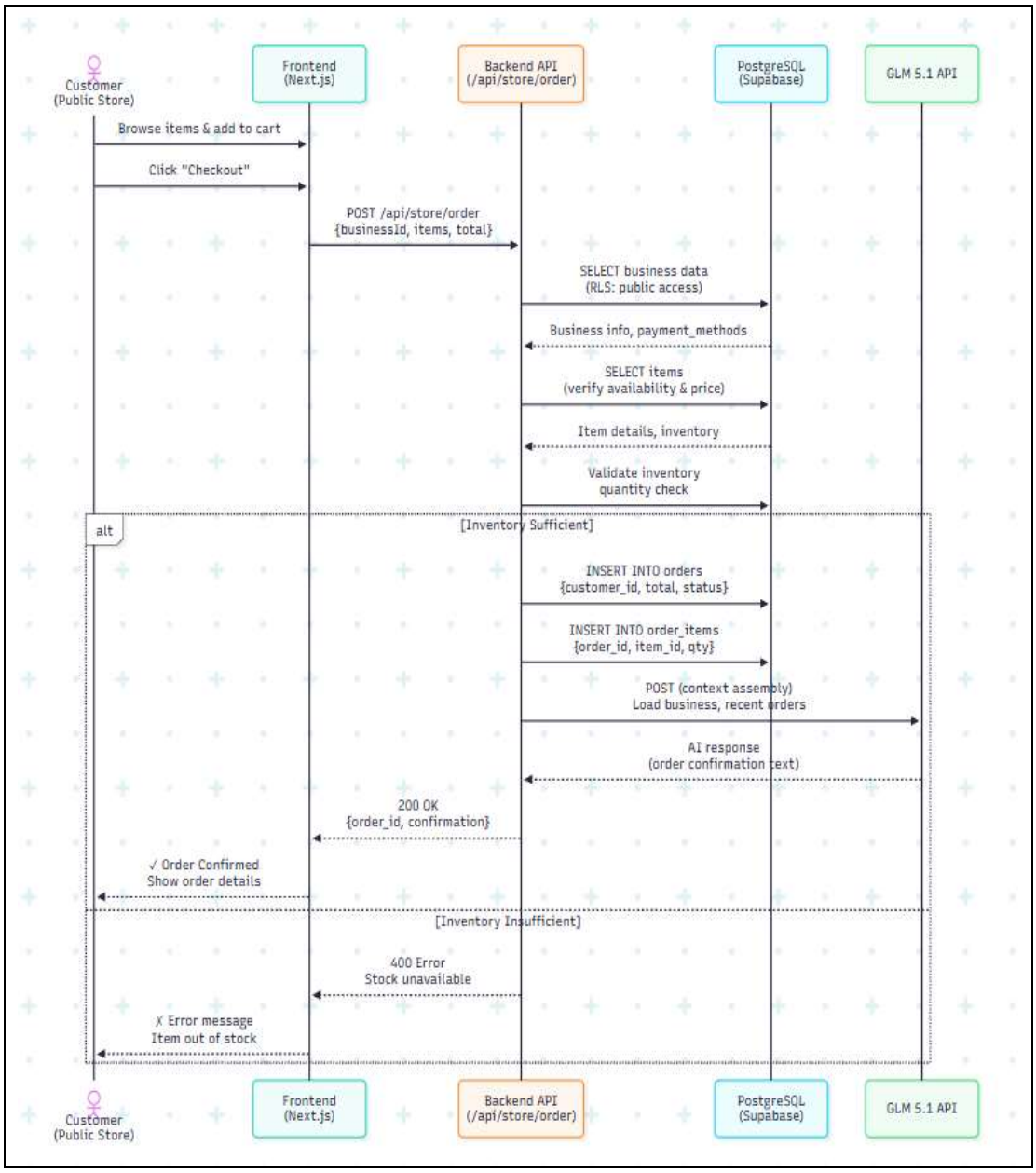


Figure 2.1.4.1(Order Flow)

Order flow shows three main interaction :

- Inventory check before persistence
- GLM integration for confirmation text
- Error handling with user feedback

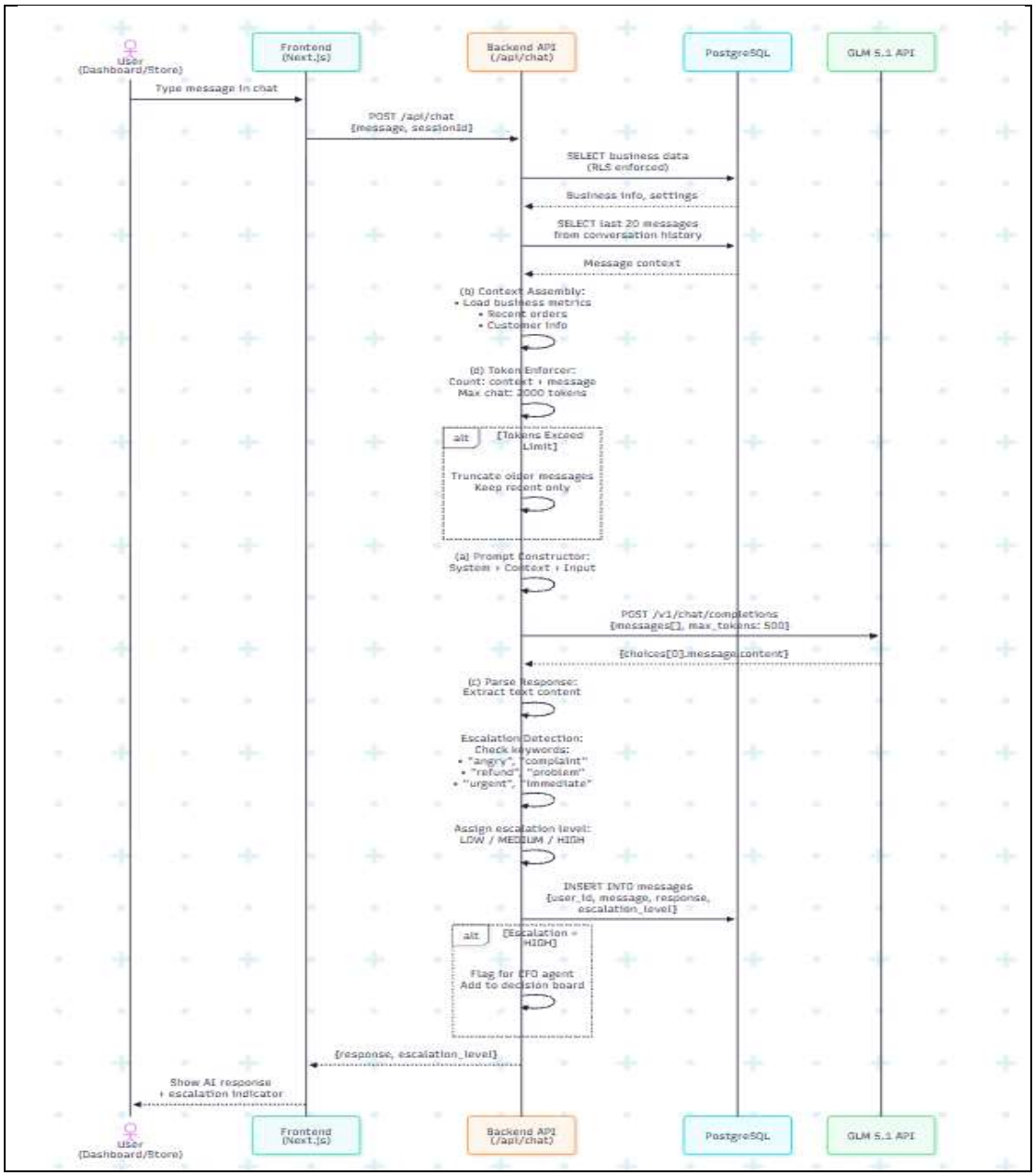


Figure 2.1.4.2 chat escalation

Figure above (2.1.4.2 chat escalation) shows four main interaction

- Context assembly from DB
- Token enforcement & truncation
- Escalation detection logic
- Response parsing & storage

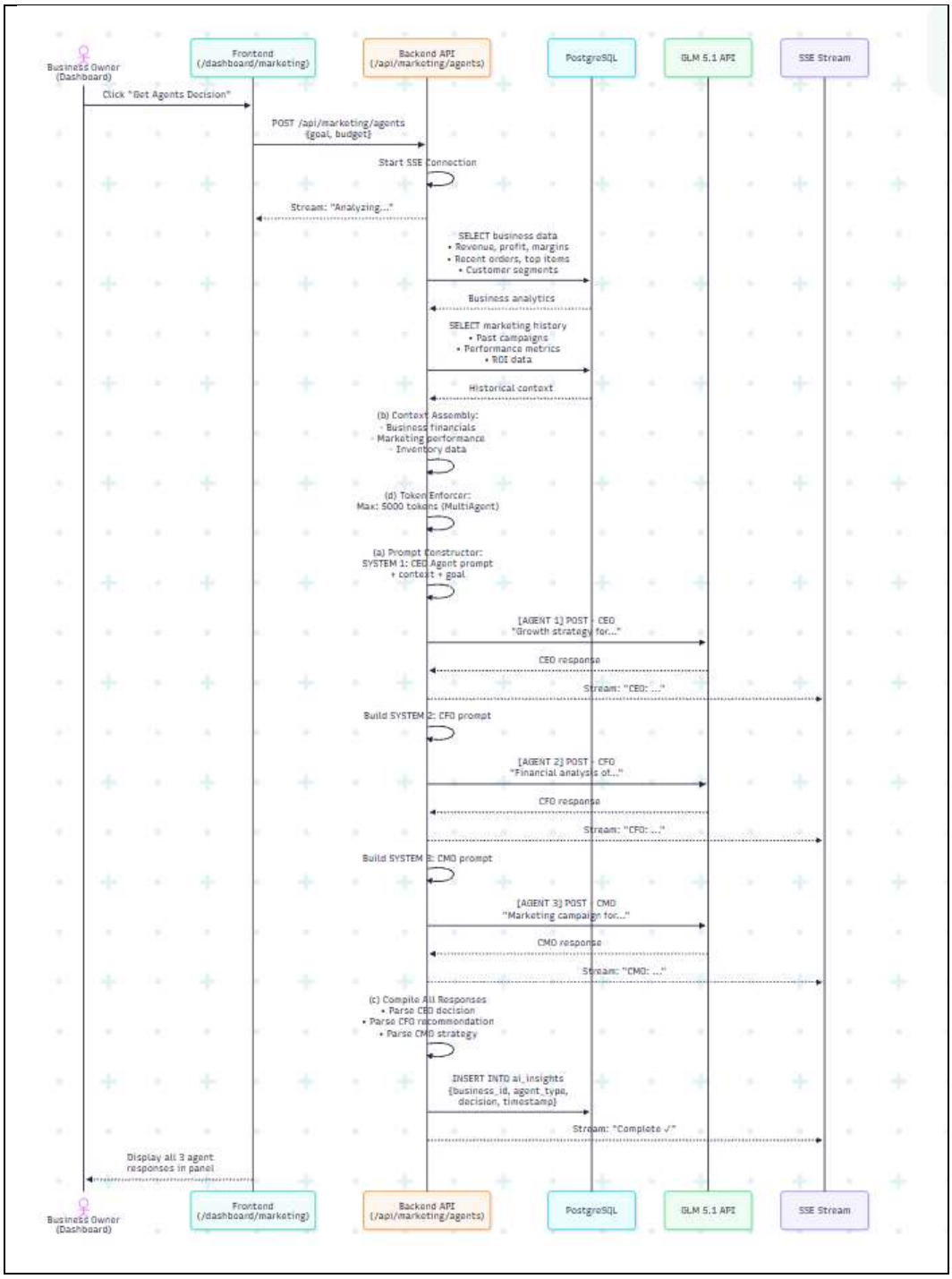


Figure 2.1.4.3 Multi-Agent Board

The figure above 2.1.4.3 Multi-Agent Board shows four main interactions:

- parallel GLM calls (CEO, CFO, CMO)
- SSE streaming for real-time updates
- Response compilation & persistence
- Context assembly with business analytics

© UMHackathon 2026

2.2 Technological Stack

LOOM's technology stack is built on cloud-native serverless architecture, leveraging managed services for scalability, reliability, and minimal operational overhead. This section details each layer's technology choices, versions, and integration rationale.

2.2.1. Frontend Technologies

Component	Technology	Version	Purpose
Framework	Next.js	14	Server-side rendering, API routes, deployment optimization
UI Library	React	18	Component-based UI, hooks, state management integration
Styling	TailwindCSS	Latest	Utility-first CSS, responsive design, rapid prototyping
State Management	Zustand	Latest	Lightweight global state, minimal boilerplate
Language	TypeScript	Latest	Type safety, IDE support, error prevention
HTTP Client	Fetch API / Axios	Native	API communication with backend

Table 2.2.1.1

Rationale:

- Next.js 14 provides automatic code splitting, API route cohabitation, and serverless deployment compatibility
- React 18 ensures modern component patterns and performance optimizations
- TailwindCSS reduces CSS bundle size and enables rapid dashboard/storefront iteration
- Zustand is chosen over Redux/Context for its minimal overhead in a lightweight app

© UMHackathon 2026

- TypeScript catches runtime errors during development, critical for multi-client handling

2.2.2 backend technologies

Component	Technology	Version	Purpose
Runtime	Node.js	18+	Serverless execution environment
Framework	Next.js API Routes	14	Serverless backend, auto-scaling, Vercel integration
Language	TypeScript	Latest	Type-safe API handlers, middleware
HTTP Method	REST API	Standard	Stateless endpoint design for scalability
Streaming	Server-Sent Events (SSE)	Standard	Real-time AI response streaming to frontend
Authentication	Supabase Auth / JWT	Standard	User session management, RLS integration
Middleware	Custom Next.js	Built-in	Request logging, error handling, CORS

Table 2.2..2.1

Rationale:

- Next.js API Routes eliminate the need for a separate backend deployment; scales automatically on Vercel
- TypeScript provides type safety across all business logic, API contracts, and database queries
- SSE chosen over WebSockets for multi-agent streaming (simpler, HTTP-based, no persistent connection overhead)
- JWT-based auth integrates natively with Supabase RLS for database-level security

© UMHackathon 2026

2.2.3 Database Technologies

Component	Technology	Version	Purpose
Database Engine	PostgreSQL	15+	ACID compliance, complex queries, JSON support
Hosting	Supabase	Cloud Managed	Managed PostgreSQL, RLS, real-time API
Infrastructure	AWS RDS	Managed	Database redundancy, backups, failover
Query Language	SQL / Supabase JS SDK	Standard	Type-safe queries, auto-generated types
Row-Level Security	PostgreSQL RLS	Native	Enforce business isolation at DB layer
Authentication	Supabase Auth	Built-in	User identity for RLS policies

Table 2.2.3.1

Rationale:

- PostgreSQL provides ACID guarantees essential for order/payment data
- Supabase RLS enforces that each business can only access its own data critical in multi-tenant architecture
- Managed service (AWS RDS) eliminates database administration overhead
- JSON columns allow flexible business configuration storage (settings, metadata)
- Real-time subscriptions (if enabled) support live dashboard updates

© UMHackathon 2026

2.2.4 External AI service

Component	Technology	Endpoint	Purpose
LLM Provider	ILMU GLM 5.1	https://api.ilmu.ai/v1/chat/completions	Core AI reasoning engine
Integration Method	REST API (POST)	Standard HTTP	Prompt submission & response retrieval
Request Format	OpenAI-compatible	messages[] format	Standardized multi-turn conversation
Response Format	JSON	{choices[0].message.content}	Structured text response parsing
Auth Method	API Key (Bearer Token)	Header-based	Service authentication
Rate Limits	Business plan dependent	Per-minute tokens	Token budget enforcement
Timeout	30-60 seconds	Configurable	Prevent hanging requests
Temperature	0.7 (default)	Adjustable	Balance creativity vs. consistency

Table 2.2.4.1

Rationale:

- ILMU GLM 5.1 selected for Malaysia-based, region-optimized LLM with multilingual support
- REST API chosen for simplicity; compatible with serverless architecture
- OpenAI-compatible format allows future model swaps without code refactoring
- JSON responses enable automated parsing for order confirmation, marketing posts, agent decisions
- Rate limiting enforced client-side (backend token counter) to optimize costs

© UMHackathon 2026

2.2.5. Hosting & Deployment

Layer	Technology	Provider	Purpose
Frontend Hosting	Vercel Edge Network	Vercel	Global CDN, automatic deployments, serverless
Backend Deployment	Vercel Serverless Functions	Vercel	Auto-scaling API routes, no infrastructure management
Database Hosting	Supabase Cloud	AWS + Supabase	Managed PostgreSQL, backups, monitoring
CI/CD	GitHub Actions / Vercel Hooks	Vercel	Automatic deployment on code push
Environment Config	.env.local / Vercel Secrets	Managed	Secure API key storage, per-environment config

Table 2.2.5.1

Rationale:

- Vercel provides end-to-end hosting: frontend + backend + auto-scaling without DevOps overhead
- Edge network ensures <100ms latency globally for business owners and customers
- Supabase abstracts database administration; automatic backups & failover
- GitHub integration enables CI/CD without manual build servers

2.2.6 External Integrations

Service	Endpoint	Purpose	Authentication
Telegram Bot API	api.telegram.org/bot{TOKEN}/	Message delivery, webhook updates	Bot token (environment variable)
Supabase Auth	auth.supabase.co	User identity, JWT tokens	API key + user credentials
Vercel API	api.vercel.com	Deployment triggers (optional)	Vercel token

© UMHackathon 2026

Table 2.2.6.1

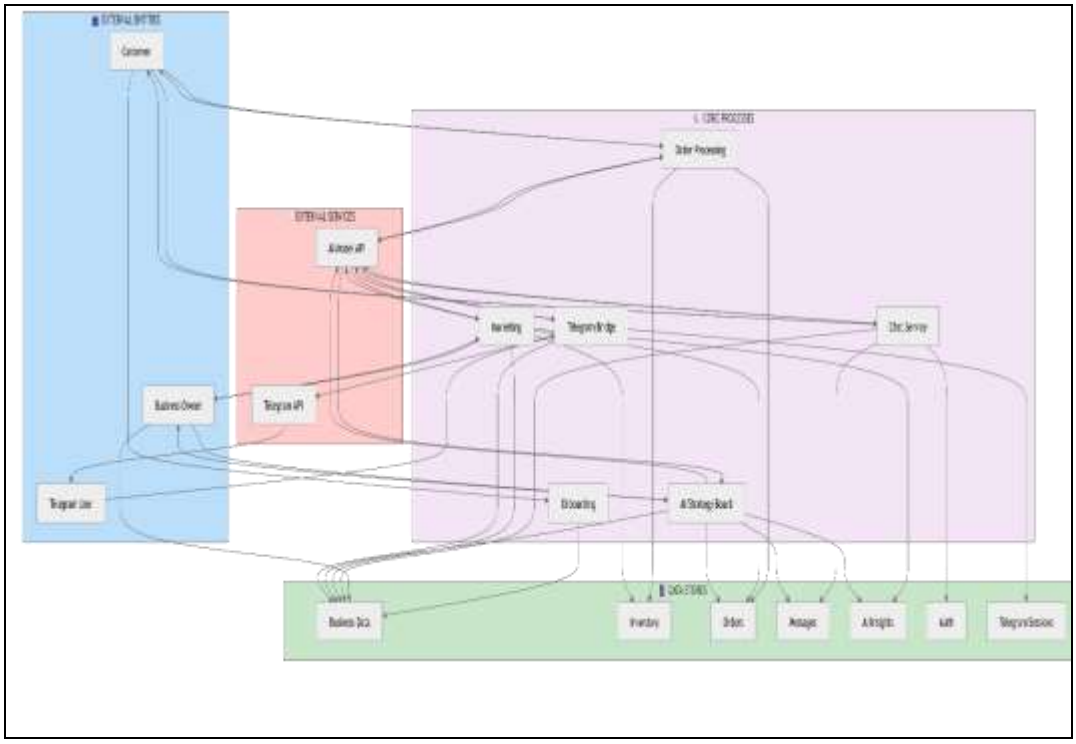
Rationale:

- Telegram Bot API chosen for v2.0 (chat-only interface, no ordering via Telegram)
- Supabase Auth provides OAuth2/password auth without building auth from scratch
- All integrations use environment-based configuration for security

2.3 Key Data Flows

2.3.1 Data Flow Diagram (DFD)

a. Illustrate the data movement across the entire system that includes external entities (users), processes, external service and databases



b. Show how proper laws are met (e.g. Black Hole)

Black Hole Principle Definition:

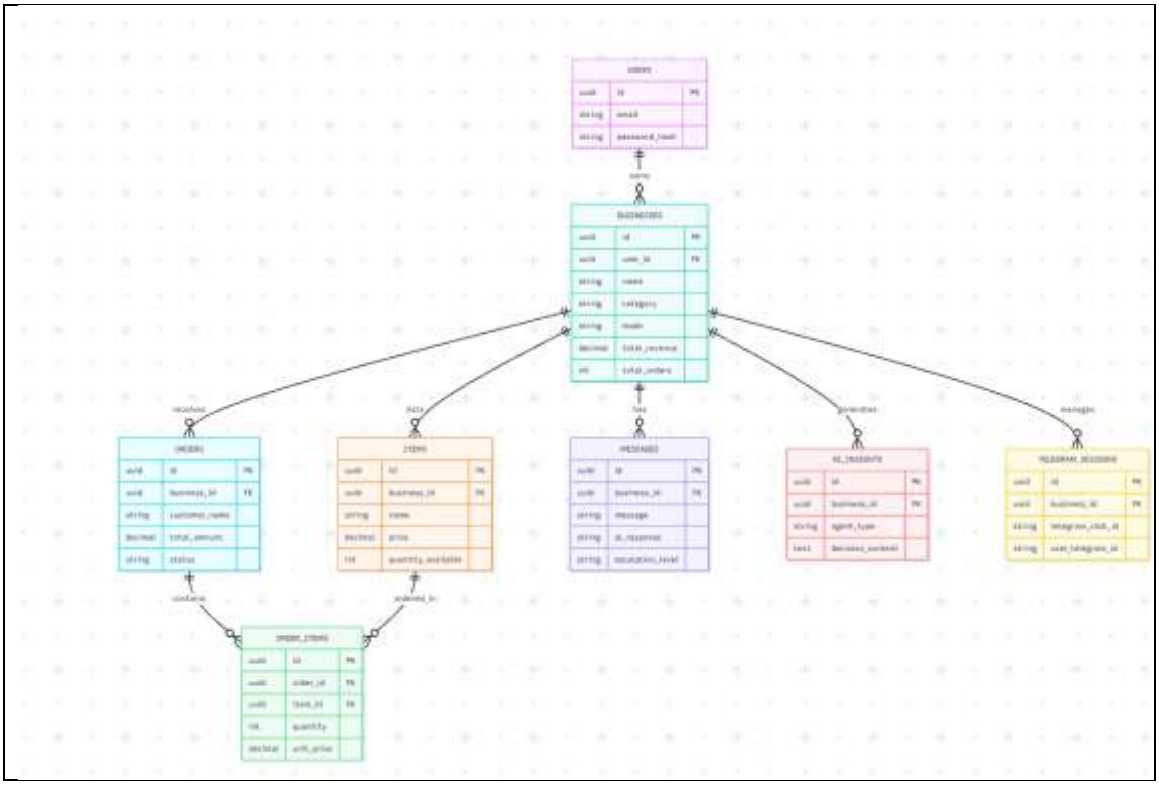
A data flow diagram violates the "Black Hole" rule if:

© UMHackathon 2026

Email: umhackathon@um.edu.my

- Data enters a process but never exits (data lost)
- Data exits a process but never entered (data created from nothing)
- Data stores exist but are never read (orphaned data)
- Data stores are written but never used (dead data)

2.3.2 Normalized Database Schema



3. Functional Requirements & Scope

This section defines the core boundaries and MVP features LOOM will demonstrate ensuring focus on high-impact functionality that showcases technical feasibility. The MVP balances autonomous business automation with AI-driven decision support.

3.1 Minimum Viable Product:

3.1.1 Below are 3-5 core features that the team is going to build and do a demo by the end of the UMHackathon 2026.

#	Feature	Description
1	Customer Order Placement via Public Store	Customers browse the business menu, select items with quantities, add them to the cart, and place orders through the public storefront. System validates inventory, calculates totals, and AI generates order confirmations. Orders persisted to database with RLS isolation.
2	AI-Powered Customer Chat Support	Customers message business via dashboard/storefront. The system uses GLM 5.1 with business context to generate intelligent responses. Escalation detection identifies angry/urgent/refund keywords and flags for owner review. Chat history retained.
3	Business Owner Dashboard	Authenticated owners view business metrics (total revenue, pending orders, inventory status). Dashboard displays recent orders, customer messages, and ability to trigger AI-generated marketing content (social media posts).
4	Multi-Agent AI Decision Board	Owner requests strategic advice; system deploys 3 parallel AI agents (CEO for growth, CFO for finance,

© UMHackathon 2026

		CMO for marketing) using GLM with business context. Agents provide independent recommendations via SSE streaming. Decisions stored as insights.

3.2 Non-Functional Requirements (NFRs)

Here, highlights the system qualities that are not features but are crucial for reliability and improve the system.

Quality	Requirement	Implementation
Scalability	LOOM must handle peak business hours (lunch 12-2pm, dinner 7-9pm) with 10x order volume spikes. API response time <500ms at p95. Support 1000+ concurrent customers across multiple businesses.	Vercel serverless auto-scales API routes instantly. Supabase managed database handles concurrent connections. Token enforcement prevents runaway GLM requests. Read replicas optional for dashboard queries.
Reliability	Order placement 99.5% uptime. Payment confirmation always creates order (no orphaned orders). Chat responses never fail silently fallback message sent on error.	Database ACID transactions ensure consistency. Supabase RLS prevents cross-business data access. Webhook verification prevents duplicate Telegram messages. Error logging + user-facing fallback responses.
Maintainability	Each service (Onboarding, Chat, Order, Marketing, Telegram, Multi-Agent) independently deployable. Code isolated per route. Configuration environment-based.	Modular Next.js API routes. Separate RLS policies per table. Environment variables for secrets (GLM key, Telegram token). CI/CD triggers only affected services on code push.
Token Latency	GLM API calls must respond within 5 seconds under normal load (p50 <2s). At p95, max 8s before timeout. User-facing loading indicator shown while awaiting response. Multi-agent board SSE streams first response within 5s.	5s timeout on GLM POST requests with fallback message. SSE streaming displays real-time progress ("CEO analyzing..."). Response parsing <500ms. Concurrent GLM calls for 3-agent board (CEO/CFO/CMO in parallel).
Cost Efficiency	Average token usage per session must not exceed 2,000 tokens. Marketing/agent board input capped at 1,000 tokens. Minimize unnecessary API calls.	Backend token counter validates before GLM call. Truncation logic removes oldest messages if >limit. Business context limited to top 5

© UMHackathon 2026

		items + last 10 orders. System prompts cached across sessions. Repeated queries cached 24 hours.

3.3 Out of Scope / Future Enhancements

Features planned for LOOM's long-term roadmap but NOT included in UMHackathon 2026 MVP:During delivery:

#	Feature	Description	Business Impact
1	Real-Time Delivery Tracking & Logistics	GPS-based automatic rider/delivery partner assignment based on location + availability. Live order tracking map for customer. Delivery time estimation and real-time notifications (order picked up, en route, arriving soon).	Completes end-to-end business automation. Differentiates LOOM as full operational platform, not just chat. Increases order fulfillment confidence and customer satisfaction.
2	Payment Gateway Integration	Direct integration with Stripe, PayPal, local e-wallets (Touch n Go, GCash). Secure payment processing, invoice generation, billing history. Real-time payment confirmation triggers order creation.	Enables actual revenue flow. Proves monetization viability. Essential for enterprise customer adoption. Currently MVP stores payment reference only.
3	Advanced AI Personalization	Personalized product recommendations per customer based on order history. Sentiment analysis on customer feedback for AI-generated review responses. Dynamic pricing based on demand, inventory levels, customer segment.	Demonstrates deep AI integration beyond generic responses. Increases average order value (AOV) through recommendations. Shows competitive advantage vs. basic chatbots.
4	Multi-Language & Localization	Auto-detect customer language via Telegram/IP geolocation. Real-time translation of menu items, business descriptions, AI responses. Localized marketing content generation (regional preferences, cultural nuances).	Unlocks ASEAN market expansion (Singapore, Thailand, Indonesia, Philippines). Shows global thinking and scalability. Critical for hackathon judges' international perspective.
5	Third-Party API & Integration Platform	Public REST API for developers. Webhook system for external integrations. Pre-built connectors with POS systems (Square, Toast), email marketing (Mailchimp, SendGrid), accounting software.	Enables ecosystem partners and developer community. Attracts enterprise customers with existing tech stacks. Creates network effect. Shows platform thinking, not just product.

Table 3.3

© UMHackathon 2026

4 Monitor, Evaluation, Assumptions & Dependencies

4.1 Technical Evaluation:

4.1.1 **Grayscale Rollout & A/B Testing**: Gradual feature deployment minimizes risk of system-wide failures. New features rolled out to percentage-based user segments with real-time monitoring.

Phase	Percentage	Duration	Monitoring Focus	Success Criteria	Next Action
Phase 0: Internal Testing	Dev team only	2 days	API response time, GLM latency, DB queries	Zero critical bugs, <100ms latency p95	Proceed to Phase 1
Phase 1: Canary Deployment	5% of businesses	24 hours	Order success rate, chat escalations, error logs	>99% order success, <2% escalation rate, <1 error/1000 requests	Proceed to Phase 2
Phase 2: Early Adopter Rollout	25% of businesses	48 hours	Revenue metrics, customer satisfaction, GLM token usage	95%+ order completion, <3s avg response time, token usage <1500 avg	Proceed to Phase 3
Phase 3: Broad Rollout	100% of businesses	Ongoing	System stability, performance degradation, cost metrics	99.5% uptime, <500ms p95 latency, cost within budget	Monitor continuously

Table 4.1.1

Monitoring Metrics per Phase:

- Phase 1 (Canary): Error rate, response latency, order abandonment rate, chat response time
- Phase 2 (Early): Revenue per business, customer churn, AI response quality, Telegram webhook failures
- Phase 3 (Broad): System-wide uptime, concurrent users, peak hour performance, cost per transaction

© UMHackathon 2026

A/B Testing Strategy (Post-Hackathon):

- Feature Variant A: Current UI (default dashboard layout)
- Feature Variant B: New UI variant (alternative dashboard layout for v2.0)
- Split Traffic: 50/50 to random users
- Metric: Time-to-order, customer engagement, support escalations
- Decision: Variant with higher engagement + lower escalations wins

4.1.2 **Emergency Rollback (ER) & Golden Release**: Critical strategy for rapid recovery during production incidents. Ensures system uptime and prevents data corruption when anomalies detected.

Golden Release Definition:
A known-stable, tested version of LOOM is deployed to production that serves as the fallback point. Stored separately and can be activated instantly if current production version fails.

- Last tested stable version (v1.0.0-stable) stored in Vercel + GitHub
- Can be deployed instantly if production fails
- Database snapshots backed up hourly via Supabase.

Emergency Rollback Triggers:

- Payment failures >5% → Trigger ER (revert to stable frontend + API routes)
- All chat/marketing requests timeout >2 min → Trigger ER
- Database connection lost >1 min → Manual ER

Automated Recovery:

If an anomaly is detected, the automated pipeline deploys the Golden Release:

- Revert frontend to v1.0.0-stable (<30s)
- Revert API routes to stable commit (<30s)
- Health check: Order flow + Chat + GLM respond?
- If stable for 2 min → Confirm production
- Post-mortem: Fix root cause, re-test, re-deploy

© UMHackathon 2026

4.1.3 Priority Matrix:

4.1.3.1 Priority Matrix

Priority	Severity	Trigger Condition	Scope	Action
P1: Critical	System Down	Order failures >5% for >1 min	All businesses	Immediate emergency rollback to Golden Release. Notify all stakeholders.
P1: Critical	Data Loss Risk	Database connection lost >1 min or RLS policy failures	All businesses	Manual emergency rollback + database restore from latest snapshot. Page on-call DBA.
P1: Critical	GLM API Down	100% of chat/marketing requests timeout for >2 min	All chat & marketing features	Auto-trigger fallback: Show message "Team will respond shortly" to all customers. Implement circuit breaker.
P1: Critical	Payment Failures	Payment confirmation errors >5% for >2 min	Checkout flow	Rollback checkout API to stable version. Disable payment temporarily; show manual payment method.
P2: High	Performance Degradation	p95 response time >2s for >5 min or Telegram webhook failures >10%	Specific routes or channels	Auto-scale Vercel functions. Check GLM rate limits. Investigate + fix within 30 min. No rollback unless P1 criteria met.
P2: High	Escalation Detection False Positive	>50% of messages flagged as HIGH escalation incorrectly	Chat service	Manual investigation: Review GLM responses. Adjust escalation keywords. Deploy fix to canary (5%) for testing.
P3: Medium	Partial Feature Failure	Telegram webhook success <95% or Marketing	Single feature/channel	Log incident. Investigate root cause within 1 hour. Deploy fix during next canary phase.

© UMHackathon 2026

		generation errors >5%		
P3: Medium	Business Context Loading Slow	Chat context assembly >1s per request	Performance	Optimize DB queries. Cache business data for 5 min. Review token limits. No immediate user impact.
P4: Low	Typos in AI Responses	Marketing post has grammar/formatting issues	Content quality	Document feedback. Improve system prompt. Deploy in next release. No urgency.

Table 4.1.3.1 Priority Matrix

Key Metrics Monitored (Real-time Dashboard):

Metric	Normal Range	Yellow Alert	Red Alert (P1)	Check Frequency
Order Success Rate	>99%	95-99%	<95% or drops 5% in 1 min	Every 30s
Chat Response Time (p95)	<2s	2-3s	>3s for >2 min	Every 30s
GLM API Response Time	<1.5s	1.5-3s	>5s or timeout	Every 1 min
Payment Error Rate	<1%	1-3%	>5% for >1 min	Every 30s
Database Query Latency	<100ms	100-500ms	>1s	Every 1 min

© UMHackathon 2026

Escalation Detection Accuracy	>95%	85-95%	<85%	Every 5 min
System Uptime	99.9%	99-99.9%	<99%	Every 1 hour

Table 4.1.3.2

Escalation path Matrix

Priority	Alert Method	Notification	Investigation Time	Action
P4 (Low)	Dashboard Log	Self-service	Within 24 hours	Document + schedule fix
P3 (Medium)	Slack Notification	On-call engineer	Within 1 hour	Investigate + deploy hotfix
P2 (High)	SMS Page	On-call engineer + TL	Within 15 minutes	Auto-scale + debug + fix
P1 (Critical)	SMS + Phone Call	All stakeholders + CEO/CTO	<5 minutes	AUTO-ROLLBACK to Golden Release

© UMHackathon 2026

4.2 Monitoring:

4.2.1 Monitoring Plan & Health Detection

Real-time system monitoring with automatic alerts when anomalies detected:

Layer	**What's Monitored**	**Tool**	**Check Frequency**	**Trigger Alert If**
API	Response time, error rate, request volume	Vercel Analytics	Every 30s	p95 >2s or error rate >1%
Database	Query latency, connection pool, RLS violations	Supabase Logs	Every 1 min	Latency >1s or failed queries >5%
GLM API	Request timeout, token usage, response quality	Custom middleware logs	Every 1 min	Timeout >5% or response time >3s
Business Logic	Order success rate, payment failures, escalation detection accuracy	App logs (Vercel)	Every 30s	Order failure >5% or payment error >2%
Infrastructure	Uptime, CPU, memory, bandwidth	Vercel dashboard	Every 5 min	Uptime <99%

Table 4.2.1

Health Check Triggers:

Anomaly Detected	Severity	Action
Response time p95 >2s for >2 min	P2	Slack alert → Auto-scale
Order failure rate >5% for >1 min	P1	SMS page + auto-rollback
GLM timeout >10% for >2 min	P1	SMS page + activate fallback

© UMHackathon 2026

Payment errors >5%	P1	Disable payment gateway + SMS
Database query lag >1s	P2	Slack alert + investigate
Escalation false positive >50%	P2	Slack alert + manual review

Table 4.2.1

4.3 Assumptions:

Key operational & environmental conditions assumed valid during LOOM development & deployment:

#	Assumption	Impact if Invalid
A1	Users have stable internet connection during order placement and chat	Order placement may fail; chats timeout; GLM responses incomplete
A2	Business owners have device access (desktop/mobile) to view dashboard	Cannot monitor orders or access AI decisions in real-time
A3	GLM API maintains 99.9% uptime during hackathon demo	All chat/marketing features fail; system becomes unusable
A4	Telegram Bot token valid and webhook endpoint accessible	Telegram customers cannot receive messages or interact
A5	Supabase database connection stable (no RLS policy failures)	Data isolation breaks; businesses may see each other's data (critical)
A6	Vercel deployment completes successfully without build errors	System cannot be deployed; demo fails
A7	Business owners provide accurate business info during onboarding	AI context incorrect; chat responses irrelevant or misleading

© UMHackathon 2026

4.4 External Dependencies:

All external services LOOM relies on, with risk assessment:

Tool	Purpose	Risk Level	Mitigation Strategy
ILMU GLM 5.1 API	Core LLM reasoning engine for chat, marketing, multi-agent decisions	HIGH	Implement fallback messages ("Team will respond soon") if GLM timeout. Rate limit enforcement to avoid quota overages.
Supabase (PostgreSQL)	Database with RLS for multi-tenant data isolation	HIGH	Hourly backups + point-in-time recovery. Verify RLS policies before production.
Vercel	Serverless hosting for frontend & API routes	MEDIUM	Use GitHub for version control. Golden Release always available for instant rollback.
Telegram Bot API	Message delivery for Telegram customers	MEDIUM	If Telegram API down, queue messages locally. Display message status to customer ("Message queued for delivery").
Supabase Auth	User authentication & JWT tokens	MEDIUM	Validate JWT tokens server-side. If auth fails, show login error (graceful degradation).
GitHub	Code repository & CI/CD integration	LOW	Local git backups. Manual deployment possible if GitHub unavailable (via Vercel dashboard).

© UMHackathon 2026

5. Project Management & Team Contributions

The duration of the Preliminary round is approximately 10 days considering the two-week sprint.

5.1 Project Timeline:

Phase	Duration	Activities	Deliverables	Owner
Phase 1: Planning & Setup	Day 1-2	Project kickoff, architecture review, tech stack setup, Vercel/Supabase provisioning, GitHub repo creation	Architecture finalized, dev environment ready, API routes scaffolded	Backend Lead
Phase 2: Core Backend & DB	Day 3-4	Database schema design, RLS policies, API routes implementation (/api/onboarding, /api/chat, /api/store), GLM integration, authentication	Database deployed with RLS, all API routes functional, GLM calls tested	Backend Lead + DB Engineer
Phase 3: Frontend & UI	Day 4-5	Dashboard design, store UI, chat interface, Telegram webhook setup, frontend components (React/Next.js)	Owner dashboard UI complete, storefront live, chat widget functional	Frontend Lead
Phase 4: AI & Integration	Day 6-7	Multi-agent board implementation (CEO/CFO/CMO), SSE streaming, escalation detection logic, Telegram bot integration, marketing content generation	3-agent board functional with streaming, Telegram messages working, marketing posts generated	AI Lead + Backend
Phase 5: Testing & QA	Day 8	End-to-end testing (order flow, chat, agents, Telegram), bug fixes, performance testing, load testing, security audit (RLS verification)	Test results, bug log cleared, performance benchmarks <2s p95, RLS verified	QA Lead + Backend
Phase 6: Deployment & Polish	Day 9	Production deployment to Vercel/Supabase, monitoring setup (alerts, health checks), Golden Release configuration, documentation finalization, demo preparation	Production environment live, monitoring dashboard active, demo script ready	DevOps Lead + PM
Phase 7: Demo & Submission	Day 10	Hackathon presentation, live demo of all 5 MVP features,	Successful demo, judges' questions	Full Team

© UMHackathon 2026

		Q&A, submission of documentation	answered, SAD + code submitted	

Table 5.1.1

Justification:

Day 1-2: 2 days ensures solid architecture planning prevents mid-sprint rework

Day 3-4: Backend foundation requires 2 days; RLS policies critical for multi-tenancy

Day 4-5: Parallel frontend work while backend integration continues (saves 1 day)

Day 6-7: AI/Telegram integration requires 2 days due to GLM API & webhook complexity

Day 8: Full QA day prevents last-minute bugs during demo

Day 9: Deployment buffer + monitoring (no demo disasters)

Day 10: Demo day with contingency time

5.2 Team Members Role:

#	Team Member	Role	Responsibilities
Person A	SHAHITTYA MD YASH SIDDQUE	Backend Lead	API route design (/api/onboarding, /api/chat, /api/store, /api/marketing), GLM integration, token enforcement, Supabase RLS policies, database optimization, scalability
Person B	AQEL ABDUL AZIZ	Frontend Lead	Dashboard UI/UX design, public storefront, chat widget, React components, TailwindCSS styling, Zustand state management, responsive design
Person C	TAJBEH KHAN JAHAN ALI	AI & Integration Lead + Project Manager	Multi-agent board implementation (CEO/CFO/CMO), escalation detection logic, Telegram webhook setup, SSE streaming, prompt engineering. Also: timeline tracking, daily standups, risk management, demo coordination

© UMHackathon 2026

Person D	AMER YOUSUF	QA, DevOps & Monitoring Lead	End-to-end testing (order/chat/agents/Telegram), bug reproduction, performance benchmarking, Vercel/Supabase deployment, monitoring setup (alerts, health checks), Golden Release configuration, RLS security validation

Table 5.2.1

Collaboration Workflow:

A ↔ B: Daily API contract reviews (endpoints, response format)
A ↔ C: GLM integration sync (Day 6 - multi-agent board)
C ↔ D: Demo preparation (Day 9 - script, talking points)
All: Daily 15-min standup (9:00 AM), blockers + progress

Workload Distribution:

Person A: 60% development (backend complex), 20% code review, 20% DevOps support
Person B: 70% development (frontend high volume), 30% UI polish
Person C: 40% development (AI focused), 30% management, 30% integration/testing
Person D: 50% testing/validation, 50% deployment/monitoring

5.3 Recommendations:

Recommendation	Benefit	Priority
Database indexing (business_id, user_id, order_date)	Query latency 500ms → 50ms	Critical
Redis cache for business context (5-min TTL)	60% fewer DB queries	High
Pre-compute system prompts	Save 200 tokens/chat, 20% cost reduction	High
API rate limiting (100 req/min per business)	Prevents abuse	Medium

© UMHackathon 2026

Circuit breaker for GLM failures	Graceful degradation	Medium
/api/health endpoint	Enables monitoring & auto-rollback	Critical
Connection pooling (max 20/business)	Prevents exhaustion	Medium

© UMHackathon 2026

Email: umhackathon@um.edu.my